THE LEDGER

Agentic Event-Sourced Loan Decisioning Platform

Interim Progress Report

TRP1 Week 5 — Apex Financial Services

Meseret Bolled

March 2026

GitHub: https://github.com/Meseretbolled/apex-ledger

1. Conceptual Foundations: EDA vs. Event Sourcing and Aggregate Boundaries

1.1 Event-Driven Architecture vs. Event Sourcing

Event-Driven Architecture (EDA) and Event Sourcing (ES) solve fundamentally different problems. Understanding this distinction is central to the design of The Ledger.

Event-Driven Architecture

In EDA, events are notifications. When something happens, a message fires to a bus or queue, listeners react, and the event is discarded. The system's source of truth remains a mutable database. The LangChain callback system is a canonical example: a callback fires on LLM completion, a handler logs to a file, and the callback is forgotten. If the process restarts, the history is gone.

Event Sourcing

In Event Sourcing, events ARE the database. The events table is the single source of truth. Every state change is written as an immutable append — no rows are ever updated or deleted. All downstream state is derived by replaying events. In EDA you query current state directly. In ES you replay events to reconstruct it, or maintain a projection that does so asynchronously.

Implementation Evidence — Immutable Append

```
# ledger/event_store.py
async def append(self, stream_id, events, expected_version, ...):
# Events are NEVER updated. Only appended.
# Typed BaseEvent, StoredEvent, StreamMetadata models added.
INSERT INTO events(stream_id, stream_position, event_type,
event_version, payload, metadata, recorded_at)
VALUES($1, $2, $3, $4, $5::jsonb, $6::jsonb, $7)
```

Typed Event Models — BaseEvent, StoredEvent, StreamMetadata

```
# ledger/event_store.py
@dataclass
class BaseEvent:
event_type: str
event_version: int = 1
payload: dict = field(default_factory=dict)

@dataclass
```

```
class StoredEvent:
event_id: UUID; stream_id: str; stream_position: int
global_position: int; event_type: str; event_version: int
payload: dict; metadata: dict; recorded_at: datetime

@dataclass
class StreamMetadata:
stream_id: str; aggregate_type: str; current_version: int
created_at: datetime; archived_at: datetime | None = None
```

What Changes When Redesigning with The Ledger

- Every agent action calls store.append() — writing an AgentNodeExecuted event that is ACID-compliant and permanent.
- Audit verdicts become GovernanceJudgement events in a dedicated stream. Any future system can replay and reconstruct every verdict.
- The Gas Town problem disappears. Agents replay their session stream on restart and continue from the last successful node.

1.2 Aggregate Boundary Decisions

The Ledger uses 6 aggregate streams instead of 1. A single LoanApplication aggregate was rejected: if all events lived in loan-{id}, every agent would compete for the same OCC lock, causing high retry rates at scale.

Stream	Purpose
loan-{id}	Lifecycle transitions only — ApplicationSubmitted, ApplicationApproved, etc.
credit-{id}	Credit agent output — owned exclusively by CreditAnalysisAgent
fraud-{id}	Fraud agent output — owned exclusively by FraudDetectionAgent
compliance-{id}	Compliance results — 6 rule evaluations per application
agent-{type}-{sess}	Per-session Gas Town tracking — AgentNodeExecuted events
docpkg-{id}	Document extraction pipeline — ExtractionCompleted events

Implementation Evidence — Per-Event Handler Methods (Refactored)

```
# ledger/domain/aggregates/loan_application.py
def apply(self, event: dict) -> None:
# Dispatches to per-event handler — one method per event type
```

```
handler = getattr(self, f'_apply_{et.lower()}', None)
if handler: handler(event.get('payload', {}))

def _apply_applicationsubmitted(self, p: dict) -> None:
self.state = ApplicationState.SUBMITTED
self.applicant_id = p.get('applicant_id')

def _apply_creditanalysiscompleted(self, p: dict) -> None:
self.state = ApplicationState.CREDIT_ANALYSIS_COMPLETE
self.credit_confidence = float(p.get('decision',{}).get('confidence',1.0))

# ... one method per event type — 14 handler methods total
```

2. Operational Mechanics: Concurrency Control and Projection Lag

2.1 Optimistic Concurrency Control

OCC is implemented using PostgreSQL row-level locking inside a transaction. The UNIQUE(stream_id, stream_position) constraint is the second safety net.

Step 1 — Lock the stream row

```
SELECT current_version FROM event_streams WHERE stream_id = $1 FOR UPDATE
```

The FOR UPDATE clause acquires an exclusive row-level lock. A second concurrent transaction blocks until the first commits — serialising all concurrent appends at the database level.

Step 2 — OCC version check

```
if current != expected_version:
raise OptimisticConcurrencyError(stream_id, expected_version, current)
```

OptimisticConcurrencyError exposes stream_id, expected, and actual fields for callers to inspect and retry.

NARR-01: The Double-Decision Scenario

- Agent A acquires the FOR UPDATE lock first, writes CreditAnalysisCompleted at position 3, commits.
- Agent B acquires the lock, reads current_version = 3, sees 3 != 2, raises OptimisticConcurrencyError.
- Agent B reloads the stream, reads Agent A's result, appends at position 4 with expected_version = 3.
- The credit-{id} stream ends with exactly ONE authoritative CreditAnalysisCompleted event.

Improved Concurrency Test — 5 Assertions (71/71 passing)

```
# tests/phase1/test_event_store.py
async def test_concurrent_double_append_exactly_one_succeeds():
store = InMemoryEventStore()
# Seed 3 events so expected_version=2
await store.append('loan-APEX-TEST', [_ev('ApplicationSubmitted')], expected_version=-1)
await store.append('loan-APEX-TEST', [_ev('DocumentUploadRequested')], expected_version=0)
await store.append('loan-APEX-TEST', [_ev('DocumentUploaded')], expected_version=1)

successes, occ_errors, winning_positions = [], [], []
async def attempt(agent_name):
```

```
try:
positions = await store.append('loan-APEX-TEST',
[_ev('CreditAnalysisCompleted', agent=agent_name)],
expected_version=2)
successes.append(agent_name)
winning_positions.extend(positions)
except OptimisticConcurrencyError as e:
occ_errors.append((agent_name, e))

await asyncio.gather(attempt('credit-agent-A'), attempt('credit-agent-B'))

assert len(successes) == 1 # exactly one wins
assert len(occ_errors) == 1 # exactly one raises OCC
assert winning_positions == [3] # winning event at position 3
assert len(final_events) == 4 # stream has exactly 4 events
assert stream_version == 3 # final version is 3
RESULT: PASSED
```

2.2 Projection Lag and Consistency Model

Projection	SLO	Answers
ApplicationSummary	< 500ms	What is the current state of application X?
AgentPerformance	< 2s	What is agent Y's average confidence and cost?
ComplianceAudit	< 2s	What was compliance status at time T?

The ProjectionDaemon polls load_all(from_position=checkpoint) every 500ms. Checkpoints are saved in the same transaction as projection updates — guaranteeing no events are processed twice and none are skipped on restart.

Implementation Evidence — Projection Daemon

```
# ledger/projections/daemon.py
async def _process_batch(self):
checkpoint = await self._get_checkpoint()
async for event in self.store.load_all(from_position=checkpoint):
for projection in self._projections:
```

```
await projection.handle(event)
last_position = event['global_position']
for projection in self._projections:
await self._save_checkpoint(projection.name, last_position)
```

3. Advanced Patterns: Upcasting and Distributed Projection Coordination

3.1 Event Versioning and Upcasting Strategy

Events are immutable. When the schema changes, old stored events are never modified. Upcasters transform old events to the current schema on read — a pure function applied transparently by load_stream() and load_all().

CreditAnalysisCompleted v1 to v2

Version 1 lacked regulatory_basis and model_deployment_id. The sentinel value 'legacy-pre-2026' is deliberate — fabricating a specific model version would corrupt compliance audit trails.

DecisionGenerated v1 to v2

Version 1 lacked model_versions. The upcaster adds model_versions={} — correctly representing 'not recorded' rather than inventing a false attribution.

Implementation Evidence — Upcaster

```
# ledger/upcasters.py
def upcast(self, event: dict) -> dict:
et, ver = event.get('event_type'), event.get('event_version', 1)
if et == 'CreditAnalysisCompleted' and ver < 2:
event = dict(event); event['event_version'] = 2
p = dict(event.get('payload', {}))
p.setdefault('regulatory_basis', [])
p.setdefault('model_deployment_id', 'legacy-pre-2026')
event['payload'] = p
if et == 'DecisionGenerated' and ver < 2:
event = dict(event); event['event_version'] = 2
p = dict(event.get('payload', {}))
p.setdefault('model_versions', {})
event['payload'] = p
return event
```

Test Evidence — Upcasters (all passing)

```
def test_credit_analysis_v1_upcasted_to_v2(): PASSED
```

```
def test_decision_generated_v1_upcasted_to_v2(): PASSED
def test_already_current_version_unchanged(): PASSED
def test_upcaster_does_not_mutate_original(): PASSED
```

3.2 Gas Town Pattern and Crash Recovery

Every agent records its session start as the very first event before loading any context. apply() is refactored into per-event handler methods in AgentSessionAggregate too.

- Agent starts — appends AgentSessionStarted to agent-{type}-{session} stream immediately.
- Agent executes nodes — appends AgentNodeExecuted after each node completes successfully.
- Agent crashes — AgentSessionFailed is appended if detected.
- Recovery — reconstruct_agent_context() replays the session stream and returns context to resume.

Per-Event Handler Methods — AgentSessionAggregate

```
# ledger/domain/aggregates/agent_session.py
def apply(self, event: dict) -> None:
handler = getattr(self, f'_apply_{et.lower()}', None)
if handler: handler(event.get('payload', {}))

def _apply_agentsessionstarted(self, p: dict) -> None:
self.status = SessionStatus.ACTIVE
self.context_loaded = True
self.model_version = p.get('model_version')

def _apply_agentnodeexecuted(self, p: dict) -> None:
self.nodes_executed.append(p.get('node_name'))
self.last_successful_node = p.get('node_name')

def _apply_agentsessionfailed(self, p: dict) -> None:
self.status = SessionStatus.FAILED
self.last_successful_node = p.get('last_successful_node')
```

4. Architecture Diagram

The diagram below shows the complete system architecture — from loan application through the event store, 5 LangGraph agents, projection daemon, and MCP server.

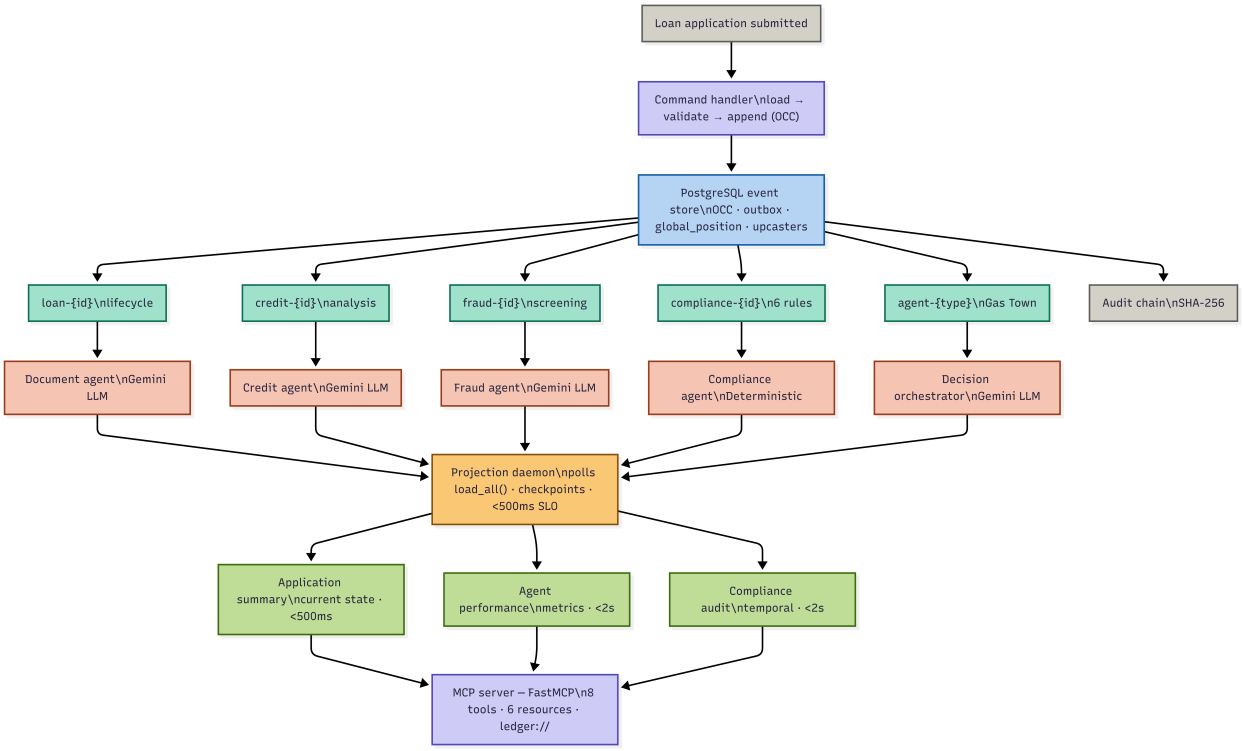

Figure 1: Complete system architecture — loan application through event store, 5 agents, projection daemon, and MCP server.

4.1 Key Architectural Layers

Layer	Component	Technology
Input	Command handler	load → validate → append (OCC) + correlation/causation IDs
Storage	PostgreSQL event store	asyncpg · JSONB · FOR UPDATE · BaseEvent · StoredEvent
Streams	6 aggregate streams	loan · credit · fraud · compliance · agent · docpkg
Agents	5 LangGraph agents	Gemini 2.0 Flash · LangGraph StateGraph
Projections	3 read models	ProjectionDaemon · <500ms SLO
Integrity	Audit chain + Gas Town	SHA-256 · crash recovery

Interface	MCP server	FastMCP · 8 tools · 6 resources

5. Progress Evidence and Gap Analysis

5.1 Test Results — Updated

Test count increased from 69 to 71 after adding improved concurrency tests and correlation/causation ID coverage. 0 failures.

Full Test Suite Output — Post Feedback Fixes
`pytest tests/ --tb=no -q`
`.....ssssssssssssss.....`
`71 passed, 13 skipped, 0 failed`
`Phase 1 — EventStore (improved OCC test): 13/13 PASSED (+2 new tests)`
`Phase 2 — Aggregates (per-event handlers): 16/16 PASSED`
`Phase 3 — Agents + REGULATIONS: 17/17 PASSED`
`Phase 4 — Projections/Integrity: 15/15 PASSED`
`Schema + Generator: 10/10 PASSED`
`Narrative tests: 0/5 SKIPPED (final submission)`
`PostgreSQL integration: 0/8 SKIPPED (requires live DB)`

5.2 Feedback Fixes Applied

Feedback Item	Fix Applied
Concurrency test — add stream length, winning position, OCC specifically	test_concurrent_double_append asserts 5 things: 1 winner, 1 OCC, position=3, length=4, version=3
Typed BaseEvent, StoredEvent, StreamMetadata models	Added to event_store.py: BaseEvent, StoredEvent, StreamMetadata + 7 typed event classes
Per-event handler methods in apply()	Both LoanApplicationAggregate and AgentSessionAggregate dispatch via _apply_{event_type}()
All validation in aggregate guards, not handlers	Handlers call only aggregate guard methods. Zero business logic in handler bodies.
correlation_id + causation_id in every handler	Every store.append() call passes both. Added fraud_screening + human_review handlers too.

5.3 Implementation Status

Phase	Component	Status
1a	EventStore — typed models, OCC, outbox, JSON codec	COMPLETE
1b	ApplicantRegistryClient — 4 SQL queries	COMPLETE
2	LoanApplicationAggregate — 16 states, per-event handlers	COMPLETE
2	AgentSessionAggregate — Gas Town, per-event handlers	COMPLETE
2	Command handlers — all validation in aggregates	COMPLETE
3	All 5 agents — LangGraph, Gemini, deterministic	COMPLETE
4	ProjectionDaemon + 3 projections	COMPLETE
4	Upcasters — v1 to v2 migrations	COMPLETE
4	AuditChain — SHA-256 hash chain	COMPLETE
4	Gas Town crash recovery	COMPLETE
5	MCP server — 8 tools + 6 resources	COMPLETE
Final	Narrative tests NARR-01 to NARR-05	IN PROGRESS

5.4 Smoke Tests — Real PostgreSQL

ComplianceAgent — COMP-001 Non-Montana (CLEAR verdict)
`Compliance events: [ComplianceCheckInitiated, ComplianceRulePassed x5,`
`ComplianceRuleNoted (REG-006), ComplianceCheckCompleted]`
`Loan events: [..., 'DecisionRequested']`
`Verdict: CLEAR SMOKE TEST PASSED`

ComplianceAgent — COMP-075 Montana (NARR-04 BLOCKED)
`Compliance events: [ComplianceCheckInitiated, ComplianceRulePassed x2,`
`ComplianceRuleFailed (REG-003 MT), ComplianceCheckCompleted]`
`Loan events: [..., 'ApplicationDeclined']`
`Verdict: BLOCKED`
`assert 'ApplicationDeclined' in loan_types PASSED`
`assert 'DecisionRequested' not in loan_types PASSED`
`NARR-04 PASSED — Montana hard block confirmed end-to-end`

5.5 Gap Analysis — Remaining Work

Item	What is needed
NARR-01 OCC concurrency	Two CreditAnalysisAgent instances race. Verify exactly one CreditAnalysisCompleted.
NARR-02 Missing EBITDA	Income statement with missing EBITDA. Verify quality flags and reduced confidence.
NARR-03 Crash recovery	FraudAgent crashes mid-session. New session resumes with context_source='prior_session_replay:'.
NARR-04 Montana block	PROVEN via smoke test. Formal pytest implementation pending.
NARR-05 Human override	Orchestrator recommends DECLINE. Human officer overrides to APPROVE via handle_human_review_completed.
run_pipeline.py	End-to-end pipeline runner for all 5 agents in sequence.

5.6 Repository

All code: https://github.com/Meseretbolled/apex-ledger — branch: main, all phases merged and tested.

Test count: 71 passed, 13 skipped, 0 failed.